

THEORY OF COMPUTATION

CSA1310

1) NFA to equivalent DFA - subset construction:

Principle:

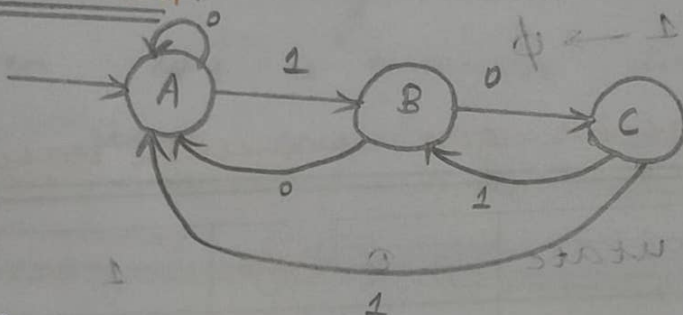
An NFA can be converted into an equivalent DFA using the subset construction Method.

* Each DFA state represents a set of NFA states.

* start state = set containing the NFA initial state.

* A DFA state is final if it contains an NFA final state.

Given NFA:



Transitions:

States : A, B, C
Alphabet : $\Sigma = \{0, 1\}$
Initial state : A
Final state : C

NFA state	0	1
A	{A}	{B}
B	{A, C}	ϕ
C	ϕ	{A, B}

Subset construction:

DFA Initial state: $\{A\}$

From $\{A\}$

$*0 \rightarrow \{A\}$

$*1 \rightarrow \{B\}$

From $\{B\}$:

$*0 \rightarrow \{A, C\}$

$*1 \rightarrow \emptyset$

From $\{A, C\}$

$*0 \rightarrow \{A\}$

$*1 \rightarrow \{A, B\}$

From $\{A, B\}$

$*0 \rightarrow \{A, C\}$

$*1 \rightarrow \{B\}$

From $\emptyset$:

$*0 \rightarrow \emptyset$

$*1 \rightarrow \emptyset$

(10)

Equivalent DFA transition table:

DFA state	0	1
$\rightarrow \{A\}$	$\{A\}$	$\{B\}$
$\{B\}$	$\{A, C\}^*$	$\emptyset$
$\{A, C\}^*$	$\{A\}$	$\{A, B\}$
$\{A, B\}$	$\{A, C\}$	$\{B\}$
$\emptyset$	$\emptyset$	$\emptyset$

$\rightarrow$ = Initial state , $*$ = Final state.

Therefore, the DFA final state is $\{A, C\}$, because it contains the NFA final state C .

Challenges in NFA $\rightarrow$ DFA conversion:

* State explosion: An NFA with n states can produce up to 2^n DFA states

* Tracking state subsets: Multiple NFA paths must be combined into a single DFA state.

* Dead state: The empty set $\emptyset$ may be required to maintain deterministic transitions.

* Final state identification: Any subset containing an NFA final state becomes a DFA final state.

Conclusion:

Thus, using subset construction, the given NFA is converted into an equivalent DFA that accepts the same language L .

2) Mathematical Induction in Automata Theory:

Introduction:

Mathematical Induction is a proof technique used to prove that a statement is true for all positive integers. In automata theory, it can be used to prove properties related to

to strings, languages, and automata.

Principle of Mathematical Induction:

To prove a statement $P(n)$ for every positive integer n , two main steps are required.

Base Case - $n = 1$:

First, substitute $n = 1$ into the given statement. If $P(1)$ is true, the statement is true for the positive integer.

Induction Hypothesis:

Assume that the statement is true for an arbitrary positive integer $n = k$.

Thus, Assume:

$P(k)$ is true

This is called Induction Hypothesis.

Induction Step - $n = k + 1$:

Now, prove that the statement is also true for $n = k + 1$.

Starting from the induction hypothesis $P(k)$, simply or manipulate the expression step-by-step until it becomes the required expression for $P(k + 1)$.

Therefore $P(k) \Rightarrow P(k+1)$

Conclusion:

Since:

1) $P(1)$ is true, and

2) $P(k) \Rightarrow P(k+1)$

by the principle of Mathematical Induction, the statement $P(n)$ is true for all positive integers n .

Application in Automata Theory:

Mathematical Induction is useful for proving:

* Properties of strings based on their length.

* Properties of languages generated by an automaton.

(V) * The number of states or transitions after repeated operations.

~~Ques~~ * Results involving concatenation and powers of strings.

Hence, mathematical induction provides a systematic way to prove that a property holds for strings or languages of every positive length.

Automaton Simulator

Test / Debug:
010

Bulk Testing

Accept (one per line):

Reject (one per line):

Automaton Simulator: DFA NFA PDA Examples

```
graph LR; start((start)) -- 0 --> start; start -- 1 --> s1((s1)); s1 -- 0 --> s2(((s2))); s1 -- 1 --> s4(((s4))); s2 -- 0 --> s2; s2 -- 1 --> s2; s2 -- 0 --> s3(((s3))); s3 -- 1 --> s2; s3 -- 1 --> s4; s4 -- 0 --> s4; s4 -- 1 --> s4;
```

Test Results:

Accept: [Empty String] -- Fail

Reject: [Empty String] -- Pass

Kyle Dickerson -- kyle.dickerson@gmail.com -- On GitHub Tools: jQuery jsPlumb Icons: FatCow